



UNIVERSIDADE D
COIMBRA

TRABALHO PRÁTICO 2

Fundamentos de Inteligência Artificial

Miguel Maria Gaspar Domingos - 2022216083
João Diogo Gaspar Domingos - 2022215704
Daniel José Paiva Nunes - 2022212535

Índice

<i>Introdução.....</i>	<i>2</i>
<i>Evolução algoritmo de aprendizagem.....</i>	<i>2</i>
Evolução função mutation.....	2
Evolução Crossover	3
Evolução Parent Selection	5
Evolução função objetivo	6
<i>Evolução de parâmetros evolutivos</i>	<i>8</i>
Parâmetros	9
<i>Vento</i>	<i>10</i>
Evolução do algoritmo de aprendizagem com vento	10
<i>Resultados.....</i>	<i>11</i>
Sem vento	11
Com vento.....	14
<i>Dificuldades.....</i>	<i>15</i>
<i>Conclusão</i>	<i>16</i>



Introdução

O presente trabalho insere-se no âmbito da unidade curricular de Fundamentos de Inteligência Artificial e tem como objetivo o desenvolvimento e aperfeiçoamento de um algoritmo de aprendizagem reforçada baseado em algoritmos evolutivos. O foco principal consiste no treino de um agente capaz de realizar uma aterragem segura e eficaz de uma nave, ajustando gradualmente os seus comportamentos com base em mecanismos de seleção, mutação, recombinação e avaliação.

Ao longo do relatório, é descrita de forma detalhada a evolução lógica dos diferentes componentes do algoritmo, nomeadamente as funções de mutação, crossover, seleção de pais e função objetivo, bem como a adaptação dos parâmetros evolutivos utilizados. Serão também apresentados os resultados obtidos nas diferentes fases do projeto, tanto em cenários sem vento como com vento, e, por fim, discutidas as principais dificuldades encontradas e as soluções que permitiram ultrapassá-las.

Evolução algoritmo de aprendizagem

Evolução função *mutation*

A função *mutation* em algoritmos evolutivos é essencial para introduzir variação genética na população, permitindo à aprendizagem máquina explorar de forma eficaz o espaço de soluções e evitar a convergência prematura em máximos locais. Ao ajustar o operador de mutação — seja gaussiano, uniforme ou adaptativo — podemos controlar o equilíbrio entre exploração global e refinamento local, otimizando o desempenho do algoritmo.

Ao longo do desenvolvimento das funções de mutação, procurámos equilibrar a exploração global e o refinamento local do espaço de soluções, adaptando a estratégia consoante o progresso do algoritmo genético. Em baixo descrevemos, em ordem cronológica, as principais versões que implementámos, culminando na função *mutation*.

No início, a função *mutation1* concretizava a ideia básica de variação genética: para cada gene somava-se um ruído gaussiano extraído de uma normal $N(0, \sigma)$. Este método era eficaz para pequenos ajustes, permitindo deslocar ligeiramente cada parâmetro e fomentar a exploração local. Porém, percebemos rapidamente que, sem qualquer controlo de limites, estes genes podiam crescer além do intervalo aceitável, corrompendo completamente a avaliação de fitness e, em alguns casos, originando comportamentos inesperados. Além disso na maioria dos processos evolutivos o algoritmo ficava preso em máximos locais, nunca atingindo zonas de fitness interessantes.

Para corrigir estas fragilidades, nasceu a *mutation2*, que manteve o núcleo gaussiano de *mutation1*, mas adicionou um simples “clamp” de valores: depois de somar o ruído, cada gene era forçado a permanecer entre -1.0 e $+1.0$. Este reforço impediu que o genótipo fugisse para regiões não interessantes, garantindo sempre que cada indivíduo se mantivesse num domínio conhecido e válido e evitando, por exemplo, falhas ou saturações na função de aptidão. Com o clamp, passámos a contar com um operador de



mutação gaussiana estável e previsível, que explorava localmente o espaço sem nunca ultrapassar fronteiras, mas que continuava a mostrar falhas na exploração mais abrangente do espaço de soluções (fixando-se muitas vezes em máximos locais).

Ainda assim, sentimos a necessidade de um mecanismo que pudesse realizar saltos bruscos, maiores quando a população estivesse presa em máximos locais, de modo a conseguir explorar novos espaços de resultados mais interessantes. Daí surgiu a *mutation3*, que abandonou totalmente o ruído gaussiano em favor de uma mutação uniforme: sempre que um gene fosse selecionado para mutação, em vez de um pequeno desvio como era feito com a mutação gaussiana, simplesmente atribuíam-se ao gene um novo valor definido uniformemente entre $[-1,1]$. Esta variante trouxe melhorias ao descobrir regiões do espaço de soluções inacessíveis pela mutação gaussiana, mas também revelou ser um pouco ruidosa demais, ao reatribuir valores inteiros, perdiam-se por vezes informação útil e boas aproximações já alcançadas.

A solução intercalar veio com a *mutation4*, um híbrido adaptativo que combinava as duas estratégias. Se o indivíduo apresentasse fitness elevado (acima de um determinado limiar, por exemplo 119, sabendo que o fitness máximo é 120), aplicava-se apenas a mutação gaussiana suave, refinando a solução em redor do ponto alcançado. Caso contrário, optava-se pela mutação uniforme clássica, garantindo saltos amplos e de modo a evitar ficar preso em mínimos locais. Esta versão melhorou a exploração e a exploração local mais precisa, mas continuava a usar o intervalo $[-1,1]$ para a parte uniforme, o que limitava o salto máximo e podia não ser suficiente para a máquina nas 100 iterações aprender o algoritmo, podendo nunca alcançar fitness perto dos 119 em tempo útil.

Por fim, consolidámos tudo na função *mutation* definitiva, que introduziu três níveis distintos de mutação uniforme e gaussiana, graduados pelo grau de fitness de cada indivíduo. Aqueles com fitness mais elevado refinavam-se localmente por meio de ruído gaussiano e clamp, polindo a solução até à perfeição. Indivíduos de fitness intermédio (115 a 119) passaram a sofrer uma mutação uniforme num intervalo alargado de $[-2,2]$, favorecendo a exploração controlada de regiões ligeiramente mais distantes. E, finalmente, os indivíduos de fitness baixo foram submetidos a uma mutação uniforme ainda mais agressiva, entre $[-3,3]$, permitindo-lhes saltar para zonas completamente novas do espaço de busca. Esta hierarquização assegura que cada membro da população recebe o tipo de mutação mais adequado ao seu desempenho: os fracos exploram amplamente, os médios testam variações moderadas e os fortes refinam por detalhes. O resultado é um algoritmo genético muito mais robusto e eficiente, capaz de manter diversidade inicial, evitar mínimos locais e, ao mesmo tempo, concentrar esforços de ajuste fino quando se aproxima de boas soluções.

Evolução Crossover

Ao longo do desenvolvimento do nosso algoritmo genético, fomos igualmente aperfeiçoando o operador do crossover para maximizar a combinação de boas características entre indivíduos e, ao mesmo tempo, preservar a diversidade genética. Tal como fizemos com a mutação, começámos por uma versão muito básica e, passo a



passo, introduzimos variações mais sofisticadas que trouxeram ganhos em convergência e robustez.

No início, utilizámos o crossover de ponto único (*crossover1*), onde escolhemos aleatoriamente uma posição no genótipo e trocávamos todas as componentes subsequentes entre dois pais para gerar um filho. Este método é extremamente simples e rápido de executar, mas mostrou-se limitado quando os genes relevantes estavam distribuídos por várias regiões do vetor: um único ponto de corte podia separar características que funcionavam bem em conjunto, degradando a qualidade dos descendentes e tornando a convergência menos estável.

Já a função de *crossover2* combina de forma adaptativa a informação genética de dois pais para gerar um descendente. Para isso tentou-se equilibrar a exploração e o refinamento: quando um dos pais é claramente superior ($\text{fitness} \geq 100$), executa-se um *crossover* aritmético ponderado que incorpora 90 % dos genes do pai mais apto (maior *fitness*) e 10 % do outro, assegurando a manutenção de características promissoras e mantendo alguma diversidade; nos demais casos, aplica-se um *crossover* uniforme, onde cada gene do filho é escolhido aleatoriamente de um dos pais com probabilidade 50 %, promovendo uma ampla combinação de traços e evitando máximos locais. Ao invocar `copy.deepcopy(p1)`, a função preserva toda a estrutura do primeiro pai, incluindo atributos não genéticos, antes de gerar o novo genótipo. A condição `random.random() < PROB_CROSSOVER` controla a taxa de recombinação, evitando que todo o descendente seja híbrido e permitindo a sobrevivência direta de alguns pais. Esta estratégia híbrida tira partido dos pontos fortes de cada técnica, o refinamento suave do *crossover* aritmético, e a capacidade de explorar livremente o espaço de soluções proporcionada pelo *crossover* uniforme, resultando num operador que se adapta automaticamente ao contexto de aptidão de cada par de pais e contribui para maior robustez e velocidade de convergência do algoritmo.

De seguida foi implementada a função *crossover* (*crossover3*) que usa um operador de recombinação híbrido e adaptativo que escolhe automaticamente entre três estratégias distintas consoante o nível de aptidão (*fitness*) dos dois pais (à semelhança do que acontecia no *crossover2*), de forma a equilibrar o refinamento local e exploração global do espaço de soluções. Quando ambos os pais atingem um *fitness* elevado (≥ 119), aplica-se o BLX- α *crossover* com α reduzido a 0,01 (de modo a garantir mais precisão), gerando cada gene do filho a partir de um valor uniformemente selecionado no intervalo $[\min(g_1, g_2) - \alpha \Delta, \max(g_1, g_2) + \alpha \Delta]$, onde $\Delta = |g_1 - g_2|$ é a diferença entre os genes dos pais. Esta variante do *crossover* é inspirada no método proposto por Eshelman e Schaffer para problemas de espaço contínuo e tem a vantagem de explorar ligeiramente além do segmento entre os genes, mantendo alta precisão ao melhorar soluções promissoras e evitando derivações bruscas desnecessárias.

Se apenas um dos pais se destacar ($\text{fitness} \geq 100$ contra $\text{fitness} < 100$), a função recorre a um *crossover* aritmético ponderado, em que 90 % do gene mais apto é misturado com 10 % do gene do outro progenitor. Esta combinação convexa privilegia a herança de traços comprovadamente eficazes, refinando-os suavemente, ao mesmo tempo que introduz variação mínima para preservar diversidade genética. Técnicas deste tipo são recomendadas quando se pretende acelerar a convergência em torno de boas regiões do espaço de soluções.



Nos restantes casos, isto é, quando ambos os pais têm fitness baixo, recorre-se ao *crossover* uniforme, em que cada gene do filho é escolhido aleatoriamente de um dos dois pais com probabilidade 50 %. Esta estratégia maximiza a exploração ao misturar cada posição independentemente, evitando padrões de dependência posicional e permitindo escapar de máximos locais que operadores baseados em pontos de corte poderiam reforçar.

O design adaptativo desta função traz várias vantagens. Em primeiro lugar, ao usar BLX- α em pais com alto fitness, garante-se uma maior precisão na vizinhança de soluções já refinadas, afastando-se apenas ligeiramente para manter diversidade sem comprometer qualidade já alcançada. Em segundo lugar, o *crossover* aritmético ponderado oferece um mecanismo de refinamento direcionado quando existe uma grande disparidade de aptidões entre pais, conduzindo o algoritmo a herdar maioritariamente traços do pai mais apto. Por fim, o *crossover* uniforme assegura diversidade e uma procura rápida no espaço de soluções, baralhando genes livremente para investigar novas combinações que possam revelar soluções ainda mais promissoras.

Sob o ponto de vista prático, a função é simples de implementar, pois cada um dos três métodos envolve apenas funções elementares de geração de números aleatórios e operações aritméticas, sem necessidade de estruturas de dados complexas. Além disto o valor do alfa foi alterado, tendo inicialmente sido definido como 0.05 e passando mais tarde a ter o valor de 0.01. Esta alteração levou a um maior refinamento quando a aprendizagem atinge níveis bons de sucesso. A redução do valor do alfa levou a uma maior aproximação na maioria dos casos nos agentes de valores de fitness de 120. Muitas vezes os agentes ficavam presos em valores de fitness acima de 119 não se conseguindo aproximar do 120. Com a redução do alfa a aprendizagem tornou-se mais refinada o que ajudou os agentes a alcançarem valores de fitness mais perto do 120.

Evolução Parent Selection

O operador do *parent selection* também foi sujeito a múltiplas transformações, que procuraram sempre por um compromisso entre pressão seletiva (escolher sistematicamente os melhores indivíduos) e manutenção da diversidade (evitar que apenas um pequeno grupo de soluções domine a evolução). Inicialmente, implementámos a seleção proporcional ao fitness, também designada por “roleta” ou fitness *proportionate selection*, onde cada indivíduo recebe uma fatia da roleta proporcional ao seu valor de aptidão, garantindo desta forma, que indivíduos mais aptos tenham maior probabilidade de ser escolhidos como pais. Apesar da simplicidade, esta abordagem revela-se vulnerável quando existem valores de fitness muito baixos, uma vez que as probabilidades podem tornar-se nulas ou indeterminadas (dependendo da *objective function*) esta forma de *parent selection* mostrava falhas sobretudo quando na *objective function* resultavam valores negativos ou perto de 0 que era o que acontecia na nossa primeira *objective function*, onde o melhor fitness era 0 e os valores de fitness eram negativos) .

Para superar esta limitação, passámos a usar a seleção por torneio, onde, repetidamente, escolhemos aleatoriamente um grupo fixo de k indivíduos e



selecionamos o de maior fitness para a reprodução. Este método apresenta a vantagem de funcionar mesmo com valores de fitness negativos e de permitir controlar a intensidade da pressão seletiva através do tamanho do torneio: torneios maiores favorecem a exploração de boas soluções, enquanto torneios pequenos preservam diversidade.

Em seguida, explorámos a seleção por ordenação (rank selection), onde primeiro ordenamos a população do torneio pelo fitness e depois atribuímos probabilidades de seleção com base na posição de cada indivíduo na lista ordenada, em vez de nos basearmos simplesmente no valor absoluto do fitness de cada indivíduo da população. Foi então definido um *selection_pressure* a 0.75 de modo a garantir uma maior probabilidade de seleção para os primeiros elementos da lista, para os elementos com maior fitness. Desta forma, evitámos que diferenças extremas no fitness conduzissem a uma concentração excessiva de seleção num único indivíduo, garantindo uma distribuição mais uniforme de oportunidades de reprodução e desta forma melhorando a robustez no final da evolução, quando os fitness se aproximam uns dos outros. Mais tarde ainda decidimos aumentar a *selection_pressure* para pais com fitness muito elevados (acima de 119) de modo a aumentar a probabilidade de seleção para pais com altas taxas de sucesso, aumentando a quantidade de genes bons que são mantidos em cada iteração.

Em suma, a evolução do operador de seleção de pais percorreu um caminho que partiu de técnicas simples de roleta, avançou para técnicas de torneio e de ranking. Cada etapa trouxe ganhos em termos de equilíbrio entre pressão seletiva e diversidade, estabilidade de amostragem e eficiência na convergência, refletindo a importância de escolher cuidadosamente o mecanismo de seleção para alcançar soluções de alta qualidade em algoritmos genéticos cada vez mais sofisticados.

Evolução função objetivo

Começámos por desenvolver uma função objetivo simples, a função *objective_function1* que atribui a cada estado da simulação uma pontuação de aptidão (fitness) composta por três componentes principais: um termo de penalização pela distância à plataforma de aterragem, um termo de penalização pela velocidade da nave horizontal e vertical e um bónus caso uma ou ambas as pernas toquem no solo. Concretamente, a penalização da distância é calculada como $|x|+|y||x|+|y|$, onde xx e yy representam a posição horizontal e vertical da nave relativamente ao alvo (bandeiras), enquanto a penalização de velocidade resulta da soma de $|vx|+|vy||vx|+|vy|$, com vxx e vyv sendo as componentes da velocidade em cada eixo. Ao penalizar ambas as funções, a função *objective_function1* incentiva o agente a manter a nave perto da zona de aterragem e a reduzir a velocidade antes do toque, dois critérios importantes para uma aterragem segura.

Além disto, *leg_bonus* oferece um acréscimo de 10 pontos por perna em contacto (até 20 pontos no total), reforçando comportamentos que efetivamente pousam a nave com as duas pernas no solo. A soma destes termos é depois feita, sendo que simulações com fitness mais elevado correspondem a melhores simulações e a simulações que devem



ser tidas como referência no processo de aprendizagem. Por fim, a função devolve também um booleano que indica se a aterragem foi bem-sucedida (`check_successful_landing`), permitindo separar a componente de aptidão contínua do critério binário de sucesso.

Entre as vantagens desta função, destaca-se a simplicidade: ao combinar penalizações lineares e um bônus discreto. Além disso a função é transparente e fácil de ajustar. Por outro lado, o uso de bônus direto por contacto com o solo é uma forma de *reward shaping* que acelera a aprendizagem, dando feedback claro e imediato sobre aproximações bem-sucedidas à condição de pouso. A abordagem penaliza simultaneamente dois aspetos cruciais (posição e velocidade), assegurando que a evolução não aprenda a “trucidar” um critério em detrimento do outro.

No entanto, a mesma simplicidade acarreta desvantagens. Primeiro, os pesos intrínsecos de cada termo são arbitrários e podem não refletir corretamente a importância relativa dos objetivos, levando a políticas que exploram desproporcionalmente o bônus de perna para “enganar/distorcer” o fitness sem uma aterragem realmente segura. Em segundo lugar, a função ignora variáveis críticas como o ângulo da nave e a velocidade centrípeta desta; assim, um indivíduo pode maximizar o fitness mantendo velocidade baixa e as pernas no solo, mas ao mesmo tempo ter uma orientação incorreta e uma velocidade angular elevada, criando soluções subótimas. Por fim, esta penalização linear não aborda efetivamente comportamentos extremos (por exemplo, altos impactos), nem diferencia contextos em que uma pequena infração de velocidade é mais grave do que uma grande distância lateral, o que nalguns cenários pode atrasar a convergência ou conduzir a *local optima*, ou seja máximos locais.

Em suma, `objective_function1` foi um ponto de partida razoável para um algoritmo evolutivo deste projeto, combinando penalizações e bônus de forma direta. Contudo, exigiu uma cuidadosa afinação dos pesos e uma extensão a múltiplas variáveis e a penalizações não lineares para capturar todas as nuances de um pouso seguro e eficiente.

De seguida desenvolvemos a função objetiva *objective_function2* que penalizava diretamente o fitness do agente através de parâmetros como a distância ao alvo, as velocidades horizontal e vertical da nave (assim como a função objetiva 1), o ângulo e a velocidade angular da nave. A isto somava-se um bônus fixo por cada pé em contacto com o solo e um novo bônus, maior, para um pouso bem-sucedido.

Nesta abordagem, a penalização usada era linear e separável (as componentes penalizam o resultado do fitness de forma independente, individualmente): as distâncias horizontais e verticais eram multiplicadas por um fator constante, as componentes de velocidade também eram penalizadas com outro fator, e o ângulo e a velocidade angular tinham ainda outro peso. No final, todas as penalizações eram somadas e subtraíamos ao bônus do pouso bem sucedido (+ 100 pontos) e do contacto das pernas com o chão a soma de todas as penalizações. Quanto maior o fitness melhor seria a performance do agente.

Percebemos, contudo, que esta versão tratava cada componente de forma demasiado independente. Pôr os termos a penalizarem cada componente de forma independente, não permitia captar interações entre variáveis (por exemplo, a combinação de alta



velocidade vertical com ângulo inclinado), o que podia levar a soluções que otimizassem um critério à custa de piorar outro de forma não prevista. O facto de a penalização ser linear implicava também que violações pequenas e grandes fossem tratadas de forma proporcional, sem distinção da gravidade; em muitos cenários, uma ligeira infração da velocidade pode ter consequências muito mais severas do que um pequeno desvio de posição, mas a função não o reflete. Assim, ao irmos para uma melhor versão (*objective_function*), introduzimos recompensas e penalizações baseadas em funções contínuas, em vez de apenas multiplicar a distância à base por 3, usamos diretamente a raiz quadrada da soma dos quadrados de x e y (distância euclidiana), o que dá uma medida única da distância à plataforma de pouso. Para a velocidade vertical e horizontal, também passámos a usar a distância euclidiana como parâmetro mantendo um peso, 2.0 para reforçar o impacto negativo no fitness de movimentos rápidos.

Na componente angular, diminuámos o peso da velocidade angular (de 1.0 para 0.5) visto que mudanças bruscas na orientação são menos críticas do que a orientação propriamente dita. E, em vez de um simples bónus estático por pé em contacto, introduzimos um bónus associado à estabilidade do pouso que depende de três fatores: a velocidade no momento do contato, o ângulo de inclinação e a proximidade ao centro da plataforma de pouso. Este bónus usa funções exponenciais decrescentes (por exemplo, $20 \cdot \exp(-\text{velocity})$), o que valoriza sobretudo pousos lentos, alinhados e centrados com a base, valorizando progressivamente aproximações suaves e controladas.

Por fim, mantivemos o bónus para pousos com sucesso, mas reduzimos o valor do bónus de 100 para 50 pontos, evitando que este bónus tornasse as penalizações irrelevantes em situações normais o que podia fazer com que o agente não evoluísse devidamente. O resultado é uma função objetivo que já não age apenas baseada em “cut-off” (sucesso versus não sucesso) nem por penalizações lineares independentes, mas que atribui recompensas contínuas conforme cada aspeto do pouso (distância, velocidade, ângulo e estabilidade) melhorando substancialmente a sensibilidade e guiando o agente a aprender pousos suaves, centrados e estáveis.

Evolução de parâmetros evolutivos

Ao longo de todo o desenvolvimento do algoritmo de aprendizagem foi-se experimentando diversos valores para os parâmetros evolutivos presentes no *crossover*, *mutation*, *parent_selection* e *objective_function*. Variáveis como o alfa no *crossover*, usada para especializar a aprendizagem do modelo para pais com fitness muito elevados; o tamanho dos participantes no torneio, no *parent_selection*; os valores do intervalo da mutação uniforme, usados durante a procura por soluções para fitness pequenos; os pesos das penalizações no *objective_function*, que estabeleciam uma relação de importância para as variáveis do espaço de observação; os próprios limites presentes no *crossover*, *mutation*, *parent_selection*, que definiam que algoritmo de aprendizagem usar para certos valores de fitness foram tudo parâmetros que foram ao longo do trabalho sendo testados de modo a se poder otimizar a taxa de sucesso do algoritmo, tendo desta forma os valores dos parâmetros finais resultado da



experimentação e da procura empírica por resultados melhores. Desta forma os valores presentes no código são valores que maximizam as capacidades do algoritmo, tendo um papel importante na taxa de sucesso elevada do programa.

Parâmetros

Serão agora apresentados os principais parâmetros evolutivos que foram testados e que são essenciais à boa performance do algoritmo evolutivo.

O parâmetro *tournament_size* da função *parent_selection*, determina quantos indivíduos são escolhidos aleatoriamente para competir entre si, enquanto *base_selection_pressure* estabelece a probabilidade com que o melhor candidato do torneio é efetivamente selecionado. Um torneio maior (*tournament_size* elevado) aumenta a probabilidade de incluir indivíduos de elevado fitness, aumentando a pressão da seleção e acelerando a convergência do algoritmo, mas ao mesmo tempo reduz a diversidade genética da população e eleva o risco de estagnação em máximos locais. Por outro lado, torneios menores abrem espaço para que indivíduos com fitness inferior também sejam escolhidos, preservando a diversidade e promovendo a exploração de diferentes regiões do espaço de soluções, embora seja necessário mais tempo de execução para alcançar bons resultados. Por estas razões escolhemos um tamanho de torneio (7) que é um misto que dá importância tanto à diversidade como à manutenção dos melhores pais. Já a pressão de seleção base, quando definida perto de 1, faz com que o vencedor do torneio seja quase sempre o com melhor fitness, reforçando a exploração e tornando o progresso mais rápido, mas em contrapartida pode levar a um empobrecimento precoce da população; valores mais baixos deste parâmetro promovem maior aleatoriedade na escolha, retardando a convergência e incentivando a descoberta de novas soluções e a exploração de novas soluções. O ajuste destes dois parâmetros é, portanto, fundamental para equilibrar a exploração no processo evolutivo, garantindo tanto eficiência na busca quanto robustez contra armadilhas que podem levar à estagnação em máximos locais.

Já o parâmetro α (alfa) do operador BLX- α *crossover* controla a amplitude do intervalo adicional em torno dos valores genéticos dos pais que poderá ser explorado ao criar um novo descendente. Especificamente, para cada par de genes, define-se o mínimo e o máximo entre os dois valores parentais e estende-se esse intervalo tanto para baixo como para cima por uma fração igual a α vezes o seu comprimento. Ao aumentar α , ampliamos o espaço de busca, permitindo que nasçam genes com valores mais afastados/diferentes dos pais e, assim, melhorando a exploração do espaço de soluções, o que pode ajudar a evitar de máximos locais, mas também pode atrasar a convergência ao longo de regiões promissoras. Em contraste, valores de α mais baixos restringem o intervalo de amostragem a regiões muito próximas aos genes dos pais, reforçando a precisão e a exploração local, acelerando a convergência quando já nos aproximamos de soluções de alta qualidade, mas correndo o risco de não descobrir novas direções promissoras. Por exemplo, reduzir α de 0,05 para 0,01 (algo que testamos e que levou a um aumento na taxa de sucesso) estreita o intervalo exploratório em 80 %, favorecendo refinamentos finos ao redor dos melhores indivíduos, enquanto aumentar α acima de 0,05 tornaria a procura mais diversificada e potencialmente mais resiliente contra estagnação.



As amplitudes dos intervalos usados na mutação uniforme são outros parâmetros essenciais para a boa performance do algoritmo evolutivo. A amplitude do intervalo determina diretamente o equilíbrio entre exploração e especialização do algoritmo evolutivo: um intervalo mais amplo permite saltos maiores no espaço de soluções, aumentando a probabilidade de descobrir regiões promissoras distantes e ajudando a escapar de máximos locais, mas também pode introduzir perturbações excessivas que retardam a convergência e geram bastante ruído na população. Por outro lado, um intervalo mais restrito concentra as variações em torno das soluções atuais, favorecendo a exploração local e acelerando o refinamento fino dos indivíduos, embora isto possa levar a um aprisionamento precoce em máximos locais caso não haja diversidade suficiente. Ajustar estes intervalos ao longo das gerações começando com valores maiores para promover uma ampla exploração e, gradualmente, diminuindo-o para polir as melhores soluções foi o que fizemos de modo a garantir um mix entre exploração e especialização, o que levou a uma melhoria na eficácia global do processo de aprendizagem.

Vento

Ao introduzir vento no cenário de aprendizagem, surgiram várias adversidades que tornaram o problema substancialmente mais complexo. Para começar, a componente estocástica do ambiente passou a variar não só com as ações do agente, mas também com rajadas de vento imprevisíveis, o que aumentou significativamente a variância nos resultados de cada episódio. Esta aleatoriedade adicional exigiu que o algoritmo evolutivo fosse capaz de gerar agentes mais robustos, capazes de compensar forças laterais e verticais inesperadas, o que retardou a convergência a valores de fitness interessantes e obrigou a testar algoritmos de aprendizagem com novos parâmetros evolutivos e lógicas de modo a compensar a instabilidade e a maior incerteza introduzida no ambiente.

Evolução do algoritmo de aprendizagem com vento

Para resolver os novos problemas foram realizadas pequenas alterações aos algoritmos vigentes, mas que levaram a melhorias notáveis na taxa de sucesso de aterragem da nave neste tipo de ambientes.

A primeira alteração feita foi na função *objective_function* onde se aumentou a penalização associada à velocidade horizontal e vertical da nave. Desta forma o algoritmo de aprendizagem redobrou os esforços no objetivo de manter a nave estabilizada neste tipo de ambientes e de manter a aterragem segura e lenta, algo fundamental para garantir uma aterragem mais facilitada por parte do agente. Desta forma, a penalização sobre a velocidade aumentou 5 vezes em relação à existente em ambientes sem vento o que levou a uma melhoria significativa na taxa de sucesso nas aterragens pelo agente. Apesar desta alteração a taxa de sucesso manteve-se baixa, sendo necessárias mais melhorias para chegar a valores de sucesso interessantes.

Reparamos que o algoritmo de aprendizagem tinha dificuldades em atingir valores de fitness interessantes, ficando por vezes preso em mínimos locais, ou nunca conseguindo atingir valores elevados de fitness durante as 100 gerações. Com este problema em vista



trocou-se os parâmetros evolucionários do *mutation* tendo-se aumentado os valores de procura na mutação uniforme, para baixos valores de fitness, de $[-3, 3]$ para $[-6, 6]$, o que levou a uma procura mais rápida no espaço de soluções, tendo desta forma o algoritmo atingido em média, valores de fitness maiores em menos gerações.

Apesar da melhoria descrita em cima ter trazido uma melhoria nos resultados, o algoritmo evolutivo para o cenário com vento continuou a apresentar alguns problemas no que toca à fase final de aprendizagem, para valores de fitness bastante elevados. Mesmo quando atingia valores de fitness elevados (acima de 119) o algoritmo apresentava bastantes dificuldades para se aproximar de valores de fitness mais próximos de 120. Para resolver este problema alteramos o parâmetro alfa do *crossover* tendo-o reduzido, o que levou a uma procura mais precisa do espaço de soluções para agentes com fitness acima de 119 (fitness muito elevado). Esta melhoria trouxe resultados satisfatórios, tendo aumentado a eficiência do algoritmo evolutivo e a taxa de sucesso de aterragem por parte do agente.

As restantes versões antigas dos algoritmos (*mutation*, *crossover*, *parente_selection*, *objective_function*) desenvolvidos ao longo do projeto e descritos em cima foram também testados em ambientes com vento, tendo apresentado resultados menos satisfatórios do que aqueles que foram conseguidos com o último algoritmo evolutivo desenvolvido. Além disso ainda testámos outros valores para os diversos parâmetros evolutivos, sendo aqui descrito apenas aqueles que levaram a uma melhoria clara no algoritmo evolucionário.

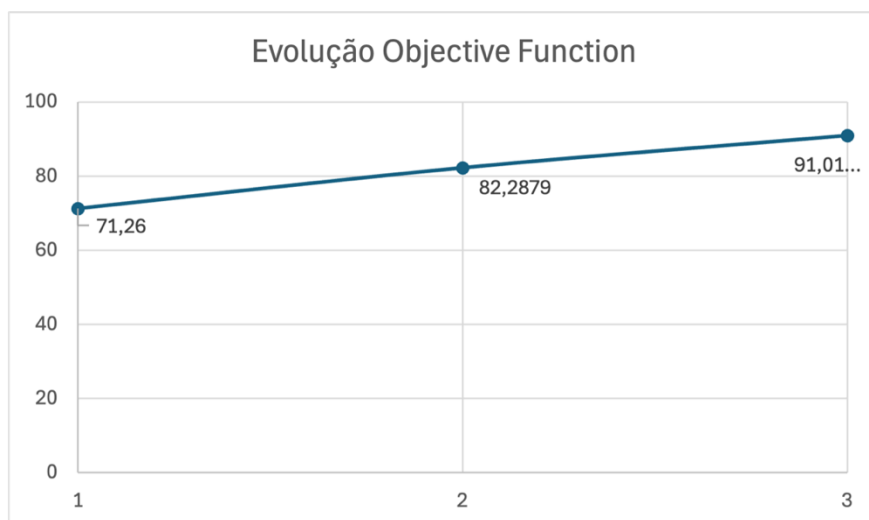
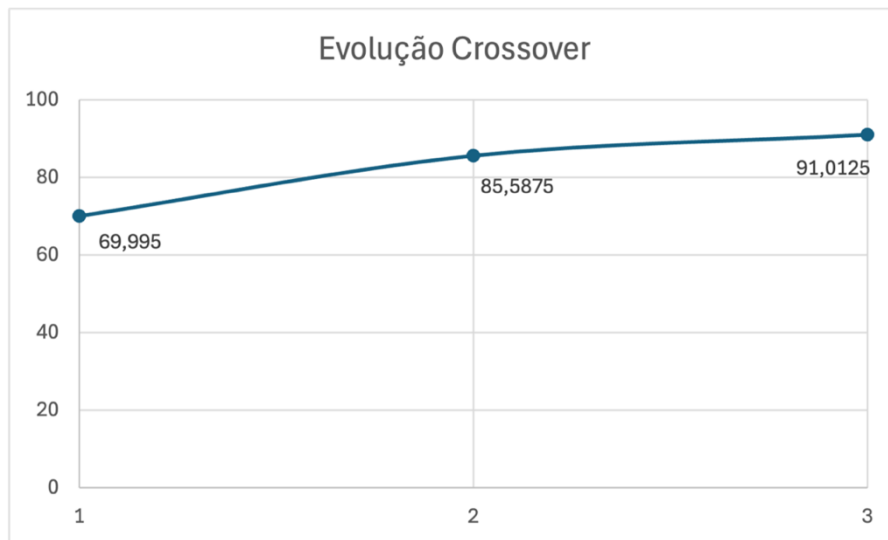
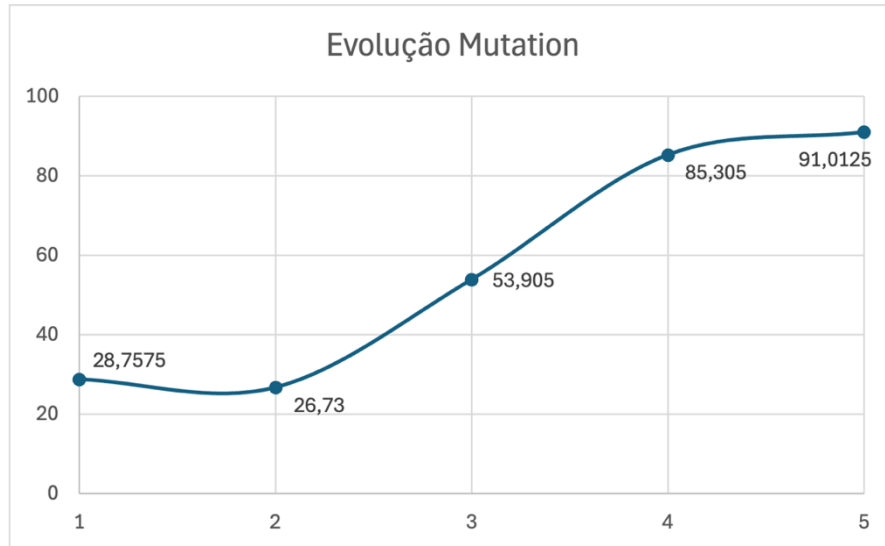
Resultados

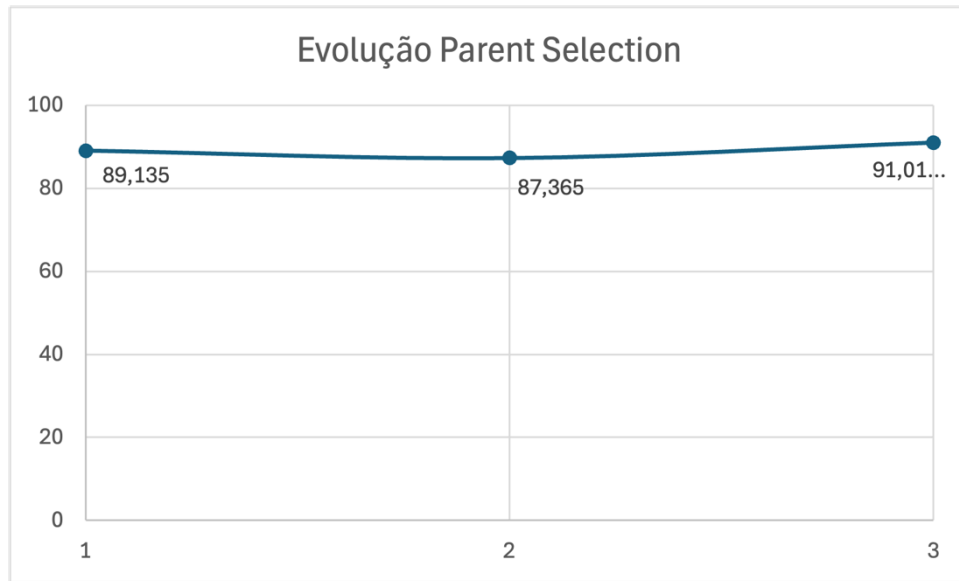
Os resultado agora apresentados mostram a evolução da taxa de sucesso de aterragem da nave ao longo do tempo consoante a evolução das diversas funcionalidades (*crossover*, *mutation*, *parent selection* e *objective function*) desenvolvidas por nós ao longo do projeto e descritas em cima. Nos gráficos apresentados, o eixo horizontal (x) refere-se à versão da funcionalidade em causa (por exemplo $x = 1$ refere-se á primeira versão desenvolvida da funcionalidade a que se refere o gráfico), enquanto o eixo vertical (y) exhibe a taxa de sucesso de aterragem em percentagem (0 –100 %). O valor da taxa de sucesso (y) resulta da média aritmética dos resultados dos oito testes presentes no enunciado do projeto, onde valores como a probabilidade de *crossover*, *mutation* e o elitismo variam. Da mesma forma, a taxa de sucesso de cada um dos oito testes foi calculada como a média aritmética dos resultados obtidos em cinco execuções para cada teste.

Sem vento

Os resultados agora aqui apresentados apresentam a performance do processo evolutivo para ambientes sem vento.







Com base nos resultados, verificámos que as alterações introduzidas nos algoritmos de evolução nem sempre conduziram a melhorias imediatas na taxa de sucesso do programa. Ainda assim, a versão mais recente de cada funcionalidade revelou-se a mais eficaz, com o algoritmo a alcançar uma taxa de sucesso na aterragem de 91 % sem vento. Além disto é notável o peso da funcionalidade *mutation* no algoritmo de aprendizagem, sendo de longe a funcionalidade que maior impacto direto tem na taxa de sucesso, revelando desta forma a importância na escolha de um bom algoritmo de mutação adequado à aprendizagem. Já o *parent selection* revelou-se a funcionalidade com menor peso na taxa de sucesso do programa, apresentando resultados parecidos de taxa de sucesso para a primeira e a última versão da funcionalidade. As funcionalidades de *crossover* e *objective function* mostraram-se importantes para uma aprendizagem mais rápida e eficiente por parte do programa.

Ao longo do projeto, foram exploradas diversas versões das funcionalidades. No entanto, os gráficos e a descrição da evolução das funcionalidades apresentados anteriormente destacam apenas os estágios mais relevantes. Durante o desenvolvimento, testaram-se outros algoritmos e ajustaram-se diferentes valores de variáveis, os quais não são aqui detalhados por se considerarem de menor relevância para os resultados finais.

Agora serão apresentados os resultados para a última versão do algoritmo evolutivo num ambiente sem vento, para os 8 casos de testes propostos pelo enunciado, assim como a média e o desvio padrão destes.

Em cada um dos oito cenários de teste propostos no enunciado, o algoritmo foi executado cinco vezes, e em cada execução calculou-se a taxa de sucesso obtida, estando esta apresentada na tabela em baixo.



TESTE/NUM TESTE	1	2	3	4	5	MÉDIA	DESVIO PADRÃO
1	86.2%	89.8%	91.7%	91.9%	84.8%	88.88%	02.89%
2	96.9%	87.4%	99.6%	70%	97.7%	90.32%	11.00%
3	84.1%	98%	95.7%	97.8%	97.7%	94.66%	05.35%
4	79.1%	99.4%	97.9%	83.7%	91.5%	90.32%	07.89%
5	75.1%	80.2%	98.7%	96.1%	95.9%	89.2%	09.62%
6	89.1%	97.6%	77.1%	77.2%	99.3%	88.06%	09.56%
7	96.6%	96.5%	78%	95%	94.4%	92.1%	07.10%
8	100%	84.8%	91.8%	99.7%	96.5%	94.56%	05.70%

Os resultados obtidos apresentam uma elevada taxa de sucesso do algoritmo evolutivo em ambientes sem vento. Contudo, em alguns testes verificou-se um desvio-padrão elevado, devido à presença de outliers que não só inflacionaram o desvio-padrão, como também afetaram a média e contribuíram para um desempenho inferior do algoritmo nesses casos.

Com vento

Os resultados agora aqui apresentados apresentam a performance do processo evolutivo para ambientes com vento.

O primeiro ponto (no eixo vertical $x = 1$) apresenta a taxa de sucesso de aterragem do agente para o código/algoritmo evolutivo inalterado. Os restantes pontos apresentam os resultados do processo evolutivo ao longo das diversas alterações realizadas nos algoritmos de evolução que foram descritos no tópico Vento.



As alterações aplicadas demonstraram-se benéficas, tendo trazido melhorias significativas na taxa de sucesso da aterragem por parte do agente. Ao aplicar o código de aprendizagem final, concebido para cenários sem vento, obtivemos uma taxa de sucesso de aproximadamente 55 % em ambientes com vento. Com as alterações nos parâmetros evolutivos atingimos uma taxa de sucesso de 67.5%, sendo notável a melhoria.

Agora serão apresentados os resultados para a última versão do algoritmo evolutivo num ambiente com vento, para os 8 casos de testes propostos pelo enunciado, assim como a média e o desvio padrão destes.

Em cada um dos oito cenários de teste propostos no enunciado, o algoritmo foi executado cinco vezes, e em cada execução calculou-se a taxa de sucesso obtida, estando esta apresentada na tabela em baixo.

TESTE/NUM TESTE	1	2	3	4	5	MÉDIA	DESVIO PADRÃO
1	75%	43.3%	65.3%	80.5%	68.8%	66.58%	12.75%
2	75%	21.6%	84.8%	69.8%	68.7%	64%	21.94%
3	61.5%	50.8%	75%	83%	67.9%	67.64%	11.06%
4	64.2%	50%	79.2%	89.9%	92.6%	75.18%	16.07%
5	81.3%	64.4%	62.8%	79.5%	76.8%	72.96%	07.79%
6	43.8%	81.8%	86.2%	47.6%	77.9%	67.46%	18.00%
7	68%	70.3%	63.8%	82.2%	86.2%	74.1%	08.60%
8	24.9%	82.9%	54.6%	97.6%	83.9%	52.78%	30.55%

Os resultados obtidos apresentam uma taxa de sucesso satisfatória, estando a redução da taxa de sucesso associada às adversidades e à aleatoriedade a que a introdução do vento no ambiente acarreta. Além disto é notável o aumento do desvio-padrão em relação aos casos de teste em ambientes sem vento. A existência de outliers intensificou-se e o algoritmo de aprendizagem mostrou maiores dificuldades para atingir níveis de fitness interessantes e aceitáveis, o que levou a resultados mais imprevisíveis e piores.

Dificuldades

Durante o desenvolvimento deste projeto, enfrentamos diversas dificuldades, sobretudo relacionadas com a escolha e afinação dos algoritmos evolutivos. Uma das primeiras barreiras foi definir uma função objetivo que refletisse de forma adequada o comportamento desejado, isto é, uma aterragem segura da nave em condições realistas. A função fornecida inicialmente era demasiado simplista, focando-se sobretudo apenas na distância final em relação à plataforma, o que levava frequentemente a soluções que



ignoravam critérios essenciais, como a orientação da nave ou o sucesso da aterragem. O ajuste dos pesos dos parâmetros também se revelou algo difícil na funcionalidade da *objective function* sendo difícil por vezes entender que parâmetros (orientação, velocidade, distancia) eram mais importantes e deveriam ser mais relevantes na pontuação final do fitness, assim como o bônus que se deveria dar para certos achievements da nave.

Outro desafio foi a parametrização do algoritmo evolutivo que se revelou um desafio crítico. A escolha de valores como alfa no *crossover* (quão preciso deveria ser o *crossover* a partir de um certo fitness), os thresholds de fitness, para que intervalos de fitness se deveria aplicar certos algoritmos de *crossover* e mutação, também os valores da mutação uniforme tiveram um impacto direto nos resultados obtidos. Parâmetros mal ajustados levaram frequentemente a populações que estagnavam rapidamente, ou, em contrapartida, a uma diversidade tão alta que a evolução se tornava instável. Esta afinação exigiu inúmeras execuções e análise dos logs para perceber quais os padrões mais promissores.

Também nos deparamos com dificuldades ao tentar encontrar algoritmos e operadores alternativos que pudessem melhorar a taxa de sucesso da aterragem. Experimentámos diferentes abordagens de seleção de progenitores (como torneios e roleta), variações no tipo de mutação (uniforme vs. Gaussiana e uma mistura de ambos) e diferentes estratégias de algoritmos de pontuação para o fitness. Embora algumas alterações tenham trazido melhorias relevantes, outras não produziram impacto significativo, o que evidenciou a natureza não linear e altamente sensível deste tipo de problema.

A escolha de algoritmos de evolução que levassem a uma melhoria geral nos 8 casos de testes fornecidos no enunciado foi um problema adicional, que levou diversas vezes a casos onde a escolha de um certo algoritmo era posta em causa (apesar de bons resultados na maioria dos testes) devido a maus resultados num dos 8 testes disponibilizado no enunciado.

Ainda, gerir a aleatoriedade do ambiente e garantir comparações justas entre configurações também foi desafiante. Para lidar com isso, utilizei múltiplas execuções com seeds controladas e baseei as conclusões em médias e variâncias estatisticamente significativas.

Por fim, a introdução de vento no ambiente trouxe adversidades acrescidas, tornando mais difícil todo o processo de aprendizagem o que se deveu sobretudo à acentuação dos problemas descritos em cima.

No geral, este projeto exigiu um equilíbrio constante entre tentativa e erro, análise crítica dos resultados e ajustes incrementais dos algoritmos, reforçando a importância de fazer experiências controladas no contexto de aprendizagem evolutiva.

Conclusão

O desenvolvimento deste projeto levou a uma melhoria na compreensão do funcionamento e das dinâmicas dos algoritmos evolutivos no contexto da reinforcement learning. Através de uma abordagem iterativa e empírica, testámos e refinámos progressivamente componentes críticas do algoritmo como a mutação, o crossover, a seleção dos progenitores/pais e a função objetivo. Estas evoluções foram essenciais



para melhorar a taxa de sucesso da aterragem da nave, tanto em ambientes controlados (sem vento) como em cenários mais desafiantes (com vento).

Os resultados demonstraram que pequenas alterações nos parâmetros e na lógica dos operadores podem ter um impacto significativo na performance do agente, evidenciando a importância da experimentação cuidadosa e da análise crítica em projetos de Inteligência Artificial. Destacou-se especialmente a importância da função de mutação e da função objetivo, que se revelaram determinantes para o sucesso do algoritmo.

Apesar das dificuldades encontradas, a gestão da aleatoriedade e a introdução de variáveis externas como o vento, com o ajuste de parâmetros e de algoritmos, conseguimos alcançar melhorias substanciais. A versão final do algoritmo mostrou-se robusta e adaptada a diferentes contextos, validando as decisões tomadas ao longo do projeto.

Em suma, este trabalho permitiu não só aplicar conceitos fundamentais da Inteligência Artificial, como também adquirir experiência prática em otimização evolutiva e no desenvolvimento de sistemas adaptativos complexos.

